Locally Checkable Problems in Distributed Computing

Dennis Olivetti

Last Update: June 3, 2026

Contents

1 Introduction **1**

2 Preliminaries **3**
- 2.1 The LOCAL model . . . 3
- 2.2 Locally Checkable Problems . . . 4
- 2.3 LCLs . . . 5
- 2.4 Distributed Complexity Theory . . . 5
  - 2.4.1 Paths and Cycles . . . 6
  - 2.4.2 Trees . . . 6
  - 2.4.3 General graphs . . . 7
- 2.5 Easy vs Hard . . . 8
- 2.6 Black-White formalism . . . 8

3 Distributed Complexity Theory **9**

4 Connections with Other Fields **10**

5 Black-White Formalism **11**

6 Round Elimination **12**

7 Other Lower Bound Techniques **13**

Chapter 1

Introduction

The goal of this book is to provide an overview of recent results about locally checkable problems in the distributed setting. Locally checkable problems are problem for which, if we are given a solution, we can efficiently check its validity. We will mainly consider the LOCAL model of distributed computing, a synchronous message-passing model, where a graph represents a communication network, and neither the size of the messages nor the computational power of the machines is restricted. The considered complexity measure is the number of communication rounds required to solve a given problem. We will address the following topics:

- Distributed Complexity Theory.
  - What are the possible distributed time complexities of locally checkable problems?
  - Does randomness help? How much? Is shared randomness more powerful than private randomness? How about quantum?
  - Can we automate our work, that is, can we automatically determine the complexity of a given problem?
- Lower Bound Techniques.
  - We will mainly talk about the round elimination technique, which has been used to prove many lower bounds over the years.
  - We will discuss other techniques, such as Mark's technique, KMW, and indistinguishability arguments.
- Connections with other fields.

- There are deep connections between the distributed setting and a mathematical field called *descriptive combinatorics*.
- There are deep connetions bewtween the distributed setting and *finitary factors of iid processes*, a topic coming from the area of analysis of randomized processes.

Chapter 2

Preliminaries

2.1 The LOCAL model

We model a network as a graph $G = (V, E)$, where the nodes represent machines, and the edges represent communication links. Nodes may be assigned unique IDs. This model works as follows:

- Time proceeds in synchronous steps (machines share a clock).
- At the beginning, each node only knows its own input and its degree. The input of a node can be, e.g., its ID, the knowledge of the size $n = |V|$ of the graph, the maximum degree Δ, and/or some additional problem-specific inputs.
- Then, computation proceeds in rounds. At each round, each node can exchange messages with its neighbors, and update its own local state (each node has its own private memory, no memory is shared between nodes).
- Each node must eventually produce an output.

The complexity of an algorithm is the worst-case number of rounds that it requires to terminate. This complexity is often measured as a function of n and Δ.

In the LOCAL model, messages can be arbitrarily large, and computation is unbounded. There are many variants of the LOCAL model that one can consider, and they may have different power:

- Is n known by the nodes? Is a polynomial upper bound on n known?

- Do nodes have IDs?
- Do nodes have access to randomness? How much randomness? A finite or infinite bit-string? Do we require Las-Vegas or Monte-Carlo algorithms?
- Messages are arbitrarily large, but are they finite? Can we e.g. send real numbers?
- Can nodes locally perform uncomputable computation?

2.2 Locally Checkable Problems

Locally Checkable Problems (LCP) are problems for which it is easy to check whether a given solution is valid. In more detail, a locally checkable problem is a problem for which there exists a constant-time distributed algorithm called *verifier* satisfying that, given a solution:

- If the solution is correct, then the algorithm outputs *yes* on all nodes.
- If the solution is incorrect, then the algorithm outputs *no* on at least one node.

Many interesting problems are locally checkable, for example:

- Coloring problems. For example, consider the $(\Delta+1)$-coloring problem, and assume that the verifier knows Δ. The verifier requires 1 round of communication, in which it checks that the color of the current node is in $\{1,\ldots,\Delta+1\}$, and that it is different from the color of the neighbors.
- Maximal independent set. The verifier needs to check that if the current node is in, then all its neighbors must be out, and that if the current node is out, then at least one neighbor is in. Observe that the *Maximum* independent set problem is not locally checkable.
- Single source shortest paths (SSSP), or at least some variants of it. Consider for example the variant in which each node outputs its own distance and a pointer to the parent. Then the verifier needs to check that the distance of the parent, plus the weight of the edge connecting the node to the parent, is equal to the distance of the node, and that no other neighbor would give a strictly smaller distance.

- Sinkless orientation, that is, orienting the edges so that each node does not have all edges oriented incoming.
- List-coloring, where each node is given a list of colors as input, and each node needs to output a color from its own list, such that the resulting coloring is proper.

2.3 LCLs

In [12], a restricted class of LCP, called Locally Checkable Labelings (LCLs), was introduced. This class is restricted enough so that we can prove interesting results, but wide enough to still contain many problems of interest. In more detail, LCLs are LCPs with the following restrictions:

- The number of input and output labels is constant (i.e., it does not depend on n).
- It is assumed that $\Delta \in O(1)$.

An example of a problem that is an LCP but not an LCL is SSSP, because nodes need to output distances, and distances could depend on n.

The nice thing about this class of problems is that each problem has a finite description:

- Let r be the number of rounds taken by the verifier. Clearly, the output of the verifier can only depend on things at distance at most r from the node.
- Since Δ and r are in $O(1)$, and the number of labels is constant, there is a finite number of graphs that the verifier could see.
- Thus, we can list the ones that the verifier would accept. Hence, we can describe a problem by listing the accepted neighborhoods.

2.4 Distributed Complexity Theory

Being LCLs so restrictive, researchers managed to prove many interesting properties about them. Moreover, many techniques that have been initially developed to understand specific LCLs, have then been generalized to understand problems that fall outside of this class (e.g., round elimination). Common questions are the following:

- What are possible LCL complexities?
- Can we automatically decide the complexity of a given LCL?
- Does shared/private randomness help?
- Does quantum help?

2.4.1 Paths and Cycles

In the case of paths and cycles, LCLs have 3 possible complexities [1, 7, 11]:

- $O(1)$. Problems with this complexity often admit very easy algorithms. In the case of directed cycles with no inputs, we even know that all problems with this complexity can be solved in 0 rounds.
- $\Theta(\log^\star n)$. Problems with this complexity require breaking symmetry, but do not require global coordination. Examples are 3-coloring and MIS.
- $\Theta(n)$ and unsolvable problems. Problems with this complexity require global coordination. An example is 2-coloring, where, by fixing the output of a single node, we are forcing the output on all other nodes.

Surprisingly, LCLs on paths and cycles cannot have any other possible complexity, and randomization never helps. This fact has the following interesting implication: you can be sloppy. Suppose you prove that your favourite problem can be solved in $O(\log n)$ rounds via a simple randomized algorithm. You automatically get that the problem can also be solved in $O(\log^\star n)$ by a more clever algorithm, deterministically.

Another surprising fact is that everything is decidable, even for LCLs with inputs. In other words, we can feed the description of an LCL to a computer program, and the program is going to output the correct complexity, and a description of an optimal algorithm.

2.4.2 Trees

In the case of trees, there are the same possible complexities of paths and cycles, plus some more [10, 2, 8]:

- $\Theta(\log n)$ deterministic. These problems can either be $\Theta(\log n)$ randomized, or $\Theta(\log \log n)$ randomized.

- $\Theta(n^{1/k})$ for any $k \in \mathbb{N}^+$. For these problems, randomness does not help.

No other complexity is possible. Moreover, it is decidable whether a problem can be solved in $O(\log n)$ or not, and in the latter case, what the exact value of k is. Summarizing, some things are still decidable, and randomness can only help exponentially or not at all, and if it helps, it only helps for problems with deterministic complexity $\Theta(\log n)$. A major open question is whether it is decidable if a problem is $\Omega(\log n)$.

2.4.3 General graphs

In general graphs, things are entirely different. For deterministic complexities, the following is known [9, 5, 2]:

- There are problems with complexity $O(1)$.
- No problem with complexity in $\omega(1) - o(\log \log^\star n)$ exists.
- The region $\Omega(\log \log^\star n) - O(\log^\star n)$ is dense: informally, for any complexity in this range, we can find a problem with a complexity that is arbitrarily near to it.
- No problem with complexity in $\omega(\log^\star n) - o(\log n)$ exists.
- The region $\Omega(\log n)$ is dense.

In the case of general graphs, undecidability results are known. Very informally, they are proved by showing that it is possible to define problems that require outputting the execution of a Turing machine, and whether the problem is constant or not depends on the running time of the machine. About randomness, a surprising connection is known: if a problem has randomized complexity $o(\log n)$, then it has randomized complexity $O(T_{\mathrm{LLL}})$, where T_{LLL} is the distributed time complexity of the constructive Lovász Local Lemma [10]. It is known that T_{LLL} is in $\Omega(\log \log n)$ and in $O(\mathrm{poly} \log \log n)$, and a big open question is determining the exact complexity of LLL. Moreover, it is known that randomness can help in different regions. For example, it is known that, for all $k \in \mathbb{N}$, there are problems with deterministic complexity $\Theta(\log^{k+1} n)$ and randomized complexity $\Theta(\log^k n \log \log n)$ [3].

2.5 Easy vs Hard

A central question about LCLs is to determine whether a problem is *easy* or *hard*, which we define as follows.

- Easy problems are problems that can be solved in $O(\log^\star n)$. We call these problems easy because, in order to solve them, we do not need to exploit structural properties of the graph, and only some basic symmetry breaking is required. In some sense, these problems are truly local.
- If a problem cannot be solved in $O(\log^\star n)$, then we know it must have deterministic complexity $\Omega(\log n)$. If a problem falls in this latter case, then we say that the problem is hard. Problems of this type cannot be solved with local information: an algorithm running in $\Omega(\log n)$ rounds (for some large-enough constant hidden in the Ω notation) is guaranteed to see at least a node of degree ≤ 2 or a cycle. Algorithms of this type are not truly local; they need to see *something* in order to be able to solve the problem. Moreover, problems of this type have randomized complexity $\Omega(\log \log n)$. Observe that this is the sweet spot to obtain high-probability guarantees: either the algorithm sees *something* in the structure of the graph that it may be able to exploit, or it sees $\Omega(\log n)$ nodes. In the latter case, consider some randomized process that has constant success probability independently on each node. By seeing $\Omega(\log n)$ nodes, the algorithm will see a successful event with high probability, which it may be able to exploit.

2.6 Black-White formalism

An important technique used to prove lower bounds in the LOCAL model is the Round Elimination technique [6]. This technique does not directly work on LCLs, but requires the problem to be described with a specific language, called black-white formalism. It turns out that, at least on trees, this is not a restriction: we can convert any given LCL into a problem in the black-white formalism, and the two problems are equivalent, up to $O(1)$ additive rounds [4]. On general graphs, problems in the black-white formalism are a strict subclass of LCLs, but most natural problems can anyway be expressed in the black-white formalism.

Chapter 3

Distributed Complexity Theory

[TODO: summarize all papers about LCLs, with proof sketches]

Chapter 4

Connections with Other Fields

[TODO: descriptive combinatorics, ffiid]

Chapter 5

Black-White Formalism

[TODO: define bw formalism, give plenty of examples]

Chapter 6

Round Elimination

[TODO: define RE, explain techniques, ...]

Chapter 7

Other Lower Bound Techniques

[TODO: Marks, KMW, indistinguishability, ...]

Bibliography

[1] Alkida Balliu, Sebastian Brandt, Yi-Jun Chang, Dennis Olivetti, Mikaël Rabie, and Jukka Suomela. The distributed complexity of locally checkable problems on paths is decidable. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 262–271. ACM, 2019.

[2] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. Almost global problems in the LOCAL model. In Ulrich Schmid and Josef Widder, editors, *32nd International Symposium on Distributed Computing, DISC 2018, New Orleans, LA, USA, October 15-19, 2018*, volume 121 of *LIPIcs*, pages 9:1–9:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[3] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. How much does randomness help with locally checkable problems? In Yuval Emek and Christian Cachin, editors, *PODC '20: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, August 3-7, 2020*, pages 299–308. ACM, 2020.

[4] Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Locally checkable labelings with small messages. In Seth Gilbert, editor, *35th International Symposium on Distributed Computing, DISC 2021, October 4-8, 2021, Freiburg, Germany (Virtual Conference)*, volume 209 of *LIPIcs*, pages 8:1–8:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[5] Alkida Balliu, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Dennis Olivetti, and Jukka Suomela. New classes of distributed time complexity. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *Proceedings of the 50th Annual ACM SIGACT Symposium on*

Theory of Computing, STOC 2018, Los Angeles, CA, USA, June 25-29, 2018, pages 1307–1318. ACM, 2018.

[6] Sebastian Brandt. An automatic speedup theorem for distributed problems. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 379–388. ACM, 2019.

[7] Sebastian Brandt, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Patric R. J. Östergård, Christopher Purcell, Joel Rybicki, Jukka Suomela, and Przemyslaw Uznanski. LCL problems on grids. In Elad Michael Schiller and Alexander A. Schwarzmann, editors, *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*, pages 101–110. ACM, 2017.

[8] Yi-Jun Chang. The complexity landscape of distributed locally checkable problems on trees. In Hagit Attiya, editor, *34th International Symposium on Distributed Computing, DISC 2020, October 12-16, 2020, Virtual Conference*, volume 179 of *LIPIcs*, pages 18:1–18:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[9] Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. An exponential separation between randomized and deterministic complexity in the LOCAL model. In Irit Dinur, editor, *IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA*, pages 615–624. IEEE Computer Society, 2016.

[10] Yi-Jun Chang and Seth Pettie. A time hierarchy theorem for the LOCAL model. *SIAM Journal on Computing*, 48(1):33–69, 2019.

[11] Yi-Jun Chang, Jan Studený, and Jukka Suomela. Distributed graph problems through an automata-theoretic lens. In Tomasz Jurdzinski and Stefan Schmid, editors, *Structural Information and Communication Complexity - 28th International Colloquium, SIROCCO 2021, Wrocław, Poland, June 28 - July 1, 2021, Proceedings*, volume 12810 of *Lecture Notes in Computer Science*, pages 31–49. Springer, 2021.

[12] Moni Naor and Larry J. Stockmeyer. What can be computed locally? *SIAM Journal on Computing*, 24(6):1259–1277, 1995.